MAINCRAFTS TECHNOLOGY

Embedded Systems Task 1

Introduction To Embedded Systems & Iot Device Design

Submitted by: Akula Pavan Venkata Durga Prasad

Internship Domain: Introduction To Embedded Systems & Iot Device Design

Project: Smart Temperature Monitoring System (Simulation)

1. Objective

The objective of this task is to understand the fundamentals of embedded systems, study common microcontrollers and sensors, and design a simple IoT-oriented prototype. The mini project monitors temperature and activates an alert when the measured value exceeds a predefined threshold.

2. Introduction to Embedded Systems

2.1 What is an embedded system?

An embedded system is an electronic system built around a processor or microcontroller to perform a specific function within a larger device. It combines hardware and software, receives input from sensors or switches, processes the input, and controls outputs such as displays, motors, LEDs, or alarms. Embedded systems are commonly designed for reliable operation, low power consumption, and real-time response.

2.2 Real-world applications

Smart homes: automatic lighting, temperature control, smart locks, and security alarms.

Automotive systems: engine control units, parking assistance, airbag controllers, and tyre-pressure monitoring.

Healthcare: digital thermometers, patient monitors, pulse oximeters, and infusion pumps.

Industrial automation: motor control, condition monitoring, process control, and equipment alarms.

Robotics: motor drivers, sensor feedback, navigation, and actuator control.

2.3 Microcontroller vs. microprocessor

Feature	Microcontroller (MCU)	Microprocessor (MPU)
Integration	CPU, memory, and peripherals usually on one chip	Primarily CPU; often needs external memory and peripherals
Typical use	Dedicated control and embedded applications	General-purpose computing and complex operating systems
Power/cost	Often lower power and lower system cost	Often higher system resources and power needs
Examples	ATmega328P, ESP32, RP2040	Processors used in PCs and many application-class computing boards

3. Component Study

3.1 Microcontroller platforms

Arduino Uno: A beginner-friendly development board commonly based on the ATmega328P. It provides digital input/output pins, analog inputs, timers, and serial communication, making it suitable for learning and small control projects.

ESP32: A microcontroller family with integrated Wi-Fi and Bluetooth on many variants. It is useful for IoT projects that send sensor readings to a phone, dashboard, or cloud service.

Raspberry Pi Pico: A compact board based on the RP2040 microcontroller. It offers GPIO, PWM, ADC, and serial interfaces and can be programmed using C/C++ or MicroPython.

3.2 Basic sensors

Temperature sensor: Measures temperature. Examples include TMP36 (analog), LM35 (analog), and DS18B20 (digital). The measurement can be used for monitoring, control, or over-temperature alerts.

Motion sensor: A PIR sensor detects changes in infrared energy caused by movement of people or animals. It is often used in security alarms and automatic lighting.

Light sensor: An LDR changes resistance depending on light intensity. With a suitable voltage-divider circuit, a microcontroller can read the light level and control lamps or displays.

4. Mini Project: Smart Temperature Monitoring System

4.1 Problem statement

A simple system is required to monitor ambient temperature and warn the user when it rises above a chosen limit. The prototype uses a simulated TMP36 temperature sensor, an Arduino Uno, and an LED as the alert indicator.

4.2 Components required

Arduino Uno (in Tinkercad Circuits)

TMP36 temperature sensor

LED and 220-ohm series resistor

Breadboard and connecting wires

Tinkercad Circuits simulation

4.3 Working principle

The TMP36 produces an analog voltage related to temperature. The Arduino reads this voltage through analog pin A0 and converts it to degrees Celsius. If the temperature is greater than or equal to the selected threshold of 30 °C, the LED connected to digital pin 8 turns on. Otherwise, the LED remains off. The threshold can be changed in the code.

4.4 Block diagram

TMP36 Temperature Sensor → Arduino Uno (analog reading and threshold comparison) → LED Alert

4.5 Circuit connections

TMP36 flat face toward you: left pin → 5 V; middle pin → A0; right pin → GND (verify the pinout shown by the simulator component).

Arduino digital pin 8 → 220-ohm resistor → LED anode (long leg).

LED cathode (short leg) → GND.

4.6 Arduino code

```
const int TEMP_PIN = A0;
const int LED_PIN = 8;
const float THRESHOLD_C = 30.0;

void setup() {
  Serial.begin(9600);
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  int adcValue = analogRead(TEMP_PIN);

  // Arduino Uno uses a nominal 5 V ADC reference and 10-bit ADC.
  float voltage = adcValue * (5.0 / 1023.0);

  // TMP36: 500 mV at 0°C, with a 10 mV/°C scale factor.
  float temperatureC = (voltage - 0.5) * 100.0;

  Serial.print("Temperature: ");
  Serial.print(temperatureC, 1);
  Serial.println(" C");

  if (temperatureC >= THRESHOLD_C) {
    digitalWrite(LED_PIN, HIGH);
    Serial.println("ALERT: Temperature threshold exceeded!");
  } else {
    digitalWrite(LED_PIN, LOW);
  }

  delay(1000);
}
```

4.7 Simulation procedure

Open Tinkercad Circuits and create a new circuit.

Add an Arduino Uno, TMP36, LED, resistor, and breadboard/wires.

Connect the components according to the connection list above.

Open the Arduino code editor, select Text, and paste the code.

Start Simulation and open the Serial Monitor.

Change the simulated TMP36 temperature. Below 30 °C, the LED should be off; at or above 30 °C, it should turn on and the alert should appear in the Serial Monitor.

Capture screenshots of the circuit, running simulation, and both below-threshold and above-threshold behaviour for the final submission.

5. Expected Results

The simulator should display the measured temperature in the Serial Monitor approximately once per second. When the temperature is below 30 °C, the alert LED remains off. When the temperature reaches or exceeds 30 °C, the LED turns on and an alert message is printed. These are expected outcomes; include your own simulation screenshots after verifying the circuit.

6. Applications and Future Improvements

Room and greenhouse temperature monitoring.

Over-temperature warning for electronic equipment.

Add a buzzer or OLED display for a local warning.

Use an ESP32 to send readings to an IoT dashboard over Wi-Fi.

Store readings over time and display a temperature graph.

7. Conclusion

This task introduces the basic concepts of embedded systems, common microcontroller platforms, and sensors. The Smart Temperature Monitoring System demonstrates the complete input-process-output cycle: the sensor measures temperature, the microcontroller converts and compares the reading, and the LED provides an alert. Simulation makes it possible to test the logic without physical hardware and provides a foundation for extending the prototype into an IoT system.

References

Arduino Documentation, https://docs.arduino.cc/

Espressif Systems, ESP32 Documentation, https://docs.espressif.com/

Raspberry Pi, Raspberry Pi Pico documentation, https://www.raspberrypi.com/documentation/microcontrollers/

Tinkercad Circuits, https://www.tinkercad.com/circuits

Analog Devices, TMP36 temperature sensor datasheet, https://www.analog.com/

Submission reminder: Replace [Your Name], run the simulation yourself, and add genuine screenshots of your working circuit before submitting. Check the full task document for any additional deliverables or formatting requirements.

TINKERDCAD CIRCUIT AND OUTPUT

